

Titan Automation Rule Model

Purpose

This document defines how automation rules operate within TitanZero.

Automation rules allow the platform to react automatically to business activity without human intervention.

Automation is executed by the Titan Pulse Engine, which listens to signals emitted by the system and performs defined actions.

Automation allows TitanZero to behave like a self-operating business system.

Automation Philosophy

TitanZero automation is built around signals.

Automation flow:

Signal occurs



Titan Pulse receives signal



Matching rules evaluated



Conditions validated



Actions executed



Signals emitted

Automation rules never bypass core engines.
All actions are executed through engine APIs.

Automation Rule Structure

Each rule contains five primary components.

trigger
conditions
actions
priority
status

These elements determine when and how automation executes.

Trigger

The trigger defines the signal that activates the rule.

Example:

trigger: work.job.completed

Triggers are always signal-based.

This ensures automation reacts to system events rather than direct database changes.

Conditions

Conditions determine whether a rule should execute after the trigger occurs.

Example conditions:

job_type = recurring
invoice_not_created = true
customer_status = active

If conditions evaluate to true, the rule proceeds to execution.

Conditions allow automation rules to be targeted and precise.

Actions

Actions define the operations that will occur when a rule executes.

Actions call APIs from core engines.

Example actions:

```
create_invoice_from_job  
send_customer_message  
schedule_follow_up_visit  
assign_quality_check
```

Each action maps to an engine method.

Example mapping:

```
create_invoice_from_job  
→ Titan Money Engine API
```

Automation rules can contain multiple actions executed in sequence.

Rule Priority

Priority determines the order in which rules execute when multiple rules respond to the same signal.

Example priorities:

```
high  
normal  
low
```

Higher priority rules execute first.

This prevents conflicting automation behaviour.

Rule Status

Rules may be enabled or disabled.

Status values:

active
paused
disabled

Paused rules allow administrators to temporarily stop automation without deleting the rule.

Example Automation Rule

Example rule:

Rule Name:
Generate Invoice After Job Completion

Trigger:
work.job.completed

Conditions:
job_billable = true
invoice_not_exists = true

Actions:
create_invoice_from_job
send_invoice_notification

Execution flow:

job.completed signal
↓
Titan Pulse detects trigger
↓
Conditions validated
↓
Invoice created
↓
Customer notified

Automation Rule Table Structure

Automation rules are stored in the following tables.

tz_automation_rules
tz_automation_triggers
tz_automation_runs
tz_automation_logs

These tables track rule definitions and execution history.

Rule Execution Records

Each rule execution produces a run record.

Example fields:

rule_id
signal_id
execution_status
started_at
completed_at
result_payload

These records allow administrators to monitor automation behaviour.

Automation Logging

Automation runs generate detailed logs.

Example log fields:

rule_id
action
result

error_message
timestamp

Logs allow developers and operators to diagnose automation issues.

Error Handling

If an automation action fails, Titan Pulse may retry execution depending on configuration.

Retry strategies may include:

retry_after_delay
retry_with_limit
escalate_to_governance

Failures may emit signals such as:

automation.failed
automation retrying
automation.escalated

Automation Escalation

When automation encounters uncertain or high-risk situations, Titan Pulse may escalate to Titan Zero governance.

Example scenario:

refund requested
↓
automation rule detects request
↓
risk level HIGH
↓
Titan Zero approval required

This ensures automation does not execute unsafe operations.

Automation Categories

Automation rules may be grouped by purpose.

Typical categories include:

- operations
- communications
- finance
- marketing
- quality_control
- system_maintenance

This grouping helps administrators manage automation more easily.

Automation Rule Builder

The Titan Command interface provides a rule builder allowing operators to create automation rules without coding.

Capabilities may include:

- selecting signal triggers

- defining conditions

- choosing actions

- testing rules

This allows business owners to customise automation behaviour.

Rule Testing

Rules may be tested using historical signals.

Example test:

simulate rule using signal work.job.completed

Testing ensures rules behave correctly before activation.

Automation Observability

The Titan Command dashboard may include an automation monitor showing:

rules executed today

failed automations

automation success rate

most triggered rules

This allows operators to understand system behaviour.

Automation Safety Rules

Developers must follow these guidelines.

1. Automation actions must call engine APIs.
2. Automation rules must not modify raw tables.
3. Automation rules must emit signals after execution.
4. Automation must respect risk classification.

5. Automation must log all executions.

Summary

The Titan Automation Rule Model enables TitanZero to operate as an automated business platform.

By reacting to signals and executing defined workflows through core engines, Titan Pulse allows the system to perform operational tasks automatically while remaining safe, observable, and governable.

-